

How To Be Awesome Like Christopher Martin

by: Christopher Martin

Last Updated: July 24, 2015

Contents

1	Logging Into Systems	2
1.1	Automating Login	3
2	Viewing Spectra	4
3	FRESCO	7
3.1	Installing FRESCO	7
3.2	Running FRESCO	8
3.2.1	Extracting Tables of cross sections	9
4	DWUCK4	11
4.1	Running a DWUCK4 file	11

Chapter 1

Logging Into Systems

There are a few ways to log into the computers downstairs. To do so, you have to first know which of the systems downstairs are available.

1. **nucgam** – Has the processed experimental data from the $^{14}\text{N}(\text{d},\text{p})$ and $^{14}\text{N}(^3\text{He},\text{d}/^3\text{He}/\alpha)$
2. **manta** – Not sure what's on here, but this system exists
3. **nucmar** – There's some *very* old code on here, older than me
4. **nucx5** – This is the new system that was setup for the scattering chamber in the "old" target room

Logging into any of the above systems is easy to do. For logging into any of the above systems, simply type in your terminal `ssh -X kirby@nucgam.physics.fsu.edu` where **nucgam** can be replaced with any of the above systems that are located downstairs. None of these can be accessed outside of NRB. After running this, it will ask you for a password, which are the following. For all of the systems besides **nucx5**, the password to log into the system is **ksq#ksq%** and for **nucx5** the password is **292415**

The data for the $^{14}\text{N}(\text{d},\text{p})$ data are found under

`/gamma10/kirby/keen320b/momotyuk/Work/7.d-p-exp/06.10.13/06.12.20/n14/16/soft/8.spectrum` with an alias being made such that if you type **melamine** in the terminal it will take you there. There is also a *copy* of this data under the home directory located at `~/Mertin_Work/14N_dp_data/` which isn't the latest but it is a backup that was made. The old (first run) $^{14}\text{N}(^3\text{He},\text{d})$ data can be found under `~/Mertin_Work/150_old/` and the newer $^{14}\text{N}(^3\text{He},\text{d})$ from January 2015 is under `~/Mertin_Work/150_Jan-2015_Data/`. To change to any of these folders, simply type `cd ~/Mertin_Work/150_Jan-2015_Data/`. In any of these folders, you can list what is in any of the folders by simply typing **ls**.

In each of the folders that house the data from the ^{15}O experiment, there's subsequent folders inside of the main folder. These other folders are labeled as the ejectiles of the ^3He reaction. For example, a folder labeled as **deuterons** means that folder falls under the $^{14}\text{N}(^3\text{He},\text{d})$ sorted data, a folder labeled as **alphas** falls under the $^{14}\text{N}(^3\text{He},\alpha)$ sorted data, etc. When in one of these folders, you can go back up one folder in the terminal by typing `cd ..` in the terminal to go up one directory.

1.1 Automating Login

If you're running a Linux computer, you can automate the login process with the program called **Expect**. To do so, simply make the following file in a directory that you'd like to store these files as such

Code 1.1: Expect login script

```
1 #!/usr/bin/expect -f
2
3 spawn ssh -X kirby@nucgam.physics.fsu.edu
4 expect "*password:" {send "PASSWORD\r"}
5 interact
```

Put this file in the folder you want to store them in, such as `~/.ssh_login/nucgam.expect` and change the permissions of the file so that it is executable by running `chmod 755 ~/.ssh_login/nucgam.expect`. From here, go into your `.bashrc` file in your home directory and add the following to the end of the file

Code 1.2: Expect alias

```
1 alias nucgam='~/.ssh_login/nucgam.expect '
```

You then need to update your terminal window so that these changes take effect, and you can do so by running `source ~/.bashrc` in the terminal that you're currently in. By doing this, you can now just type `nucgam` into your terminal and it will automatically log you into the system that you're trying to access.

Chapter 2

Viewing Spectra

The location of the data can be found in Chapter 1.

The program that I used to analyze and view spectra is GNUscope, where the manual for it can be found [here](#). GNUscope can open `.nsm`, `.spk`, and `.txt` files, with the latter being in the format of an ASCII file with the first column being the channel and the second being the counts for that channel. To open a spectra with GNUscope, simply change directory to the location of the spectra and type the following in the terminal `gnuscope -f spectra_file.txt` and a new window will open though the spectra will not be displayed. More than one spectra can be opened by typing `gnuscope -f spectra1.txt spectra2.txt` which will store both spectra into memory to be analyzed. The following are the most common utilities for analyzing spectra in GNUscope.

- Display spectra – Either at the top title bar you can go to **Display >> Display** or you can hit the letter **D** on the keyboard. A new smaller window will open which will ask you to decide on the spectra that you want to display. To do so, click on the spectra in the new menu box and then click on "Display." If you're looking at a `.txt` file, they will not have a label and you will have to enter the number of the spectra only (ignore the colon).
- Expand – To expand a region in the spectra, select the region and then type the key **E** to expand over that region.
- Background Subtraction – To average the background out, select a region of the background with the mouse and go to **Analysis >> Set Background** or type the letter **B**.
- Gaussian Fit – For a single Gaussian fit, select the region you want to fit with the mouse and go to **Analysis >> Gaussian** or type the letter **G**. For a double Gaussian Fit, simply select the double Gaussian region that you want to fit and go to **Analysis >> Double Gaussian** or hit **Shift+G** on the keyboard. *For Double Gaussian Only:* After initiating the double gaussian fit, it will ask for you to *guess* the FWHM for the two peaks so that it can try and guess the fit of each. So enter them both, where each one is separated by a single space. The fit parameters and the yield will be found in the text box at the top of the window. For the Double Gaussian fit,

there is no "done" button, so after typing in the FWHM for both peaks, simply hit **Enter** on the keyboard to do the calculation.

- **Sum Region** – To sum counts/find the yield, select the region and go to **Analysis >> Sum** or hit **S** on the keyboard.
- **Gainshift** – To gainshift spectra, go to **Manipulate >> Gainshift**, where it calls for the terms in the form of a polynomial up to the third power.
- **Normalize** – To normalize a spectra to another, go to **Manipulate >> Normalize** or type **Ctrl+N** and the input the factor that you would like to normalize the spectra by.
- **Subtract Spectra** – To subtract spectra, go to **Manipulate >> Subtract** or use **Ctrl+S**. From here, it will ask for the input in the following format **Spectrum1 - Spectrum2 + Const = Something** where **Something** can be anything. An example of this input would be **1 2 0 99**, which would subtract spectrum 2 from spectrum 1, add 0 to it, and then produce a plot.
- **Print Spectra** – You can also print from GNUscope, just go to **File >> Print** and if on Nucgam at NRB at FSU, the printer would be **ljnucmar** for the printer next to nucgam. You can also print the file to pdf by going to **File >> Print to File**
- **Write to ASCII** – To write a spectra to ASCII after manipulating it, simply go to **File >> Write Lipha** or type the letter **L**, afterwhich a menu will pop up and you can type the name that you want to save it as. However, after doing so it won't be completely readable, so use the script below to clean up the file by running it as **./LiphaCleanup.sed saved_filename**

Code 2.1: LiphaCleanup.sed script

```
1  #!/bin/bash
2
3  # To run this file , save it as "LiphaCleanup.sed"
4  # and then run "chmod 755 LiphaCleanup.sed"
5  # Then to execute simply do  "./LiphaCleanup.sed filename"
6
7  if [ $# -eq 0 ]; then
8      echo "No arguments provided. Need exactly 1 file to fix."
9      echo "Exiting..."
10     exit 1
11 fi
12
13 if [ $# -ne 1 ]; then
14     echo "Need exactly 1 argument. The file that is to be fixed."
15     echo "Exiting..."
16     exit 1
17 fi
18
19 if [ ! -f $1 ]; then
20     echo "File \"$1\" not found."
21     echo "Exiting..."
22     exit 1
23 fi
24
25 sed -i '/[0-9]* [0-9]* /g' "$@"
26 sed -i '/^$/d' "$@"
27
28 echo "Cleaned up the data in \"$1\" to plain ASCII."
```

Chapter 3

FRESCO

FRESCO is freely available software where the [mainpage](#) is here, the [documentation](#) can be found here, and the [source code](#) can be downloaded from here.

3.1 Installing FRESCO

Note: *The following steps are for bash-shell only. Other shells are similar, but not specific to bash*

First of all, a FORTRAN compiler is needed on your system. The most common one is gfortran and can be found in your systems repositories. After downloading the software, it can be unzipped with the following commands

Code 3.1: Commands to untar a file

```
1 gunzip fres-v29.tar.gz
2 tar -xvf fres-v29.tar
3 rm fres-v29.tar
```

Then, make your way to the source directory inside the new folder. Open up the makefile and change the first few lines to the following

Code 3.2: FRESCO makefile

```
1 ...
2 FFLAGS = -O2          # generic
3 # FFLAGS = -O2 -Wall -ftrace=full # generic
4 FFLAGS = -O2 -Wtabs # gfortran
5 #FFLAGS = -O2 -Wall # generic
6 #FFLAGS = -O2 -m32 # 32-bit machines generic
7 #FFLAGS = -O2 -m64 # 64-bit machines generic
8
9 #FFLAGS = -O2 -dcfuns -w # for nag f95
10 #FFLAGS = -C=all -C=undefined -dcfuns -w -O0 -gline # for nag f95 debug
11 #FFLAGS = -O2 -qmaxmem=16384 # optimisation for xlf
12 #FFLAGS = -Wall -g -fbounds-check -ftrace=full # debug for g95
13 #FFLAGS = -g -C # generic debug for f90
```

```

14 FC=gfortran
15 F90C=$(FC)
16 ...

```

Now, open up the `aliases` file in the source directory, and change it to the following (again, this is for bash)

Code 3.3: FRESKO aliases file

```

1 export FRESKOLIB=/location/to/folder/fres/
2 export PATH=$PATH:FRESKOLIB
3 alias run='chmod +x \!* ; nohup \!* >& \!*.out &'

```

From here, you must copy the `aliases` file to the following directory

```

1 cp aliases ~/.fresco.aliases

```

Now you must edit your `.bashrc` file in your home directory by adding the following to the end of it

```

1 source ~/.fresco.aliases

```

Now, navigate back to the `source` folder, and run the `make` command to compile all the code. You should end up with a `fresco` executable and an `sfresco` executable as well. Ignore the warnings during the make

We might as well make our lives running FRESKO easier. To do this, add the following to the end of your `.bashrc` file

Code 3.4: `.bashrc` FRESKO aliases

```

1 alias fresco='/directory/to/executable/file/fresco '
2 alias sfresco='/directory/to/executable/file/sfresco '

```

This makes it so you don't have to type out the full pathname when you want to run FRESKO, simply requiring you to type in `fresco` instead.

3.2 Running FRESKO

When you have your FRESKO file, you can run FRESKO calculations in the following way

```

1 /dir/to/fresco < input.fri > output.fro

```

where you can exchange `/dir/to/fresco/` with just `fresco` if you followed the above install process and made an alias for the `fresco` executable. This will output the `output.fro` file, which has all the information about the calculation, and it will also output `fort.##` files as well. An example of what these files represent can be found below.

3:local copy of User input.	6:standard output.
12:transfer kernels.	13:total cross sections/state.
15:local equivalent potentials.	16:tables of cross sections.
35:Astrophysics S-factors / Ecm.	39:cross sections for each Ecm.

40:all cross sectns. for each Elab.	46:bs wave functions & ANC ratios.
56:Fusion for each Jtotal.	58:Single-particle wfns.
75:S-factors for lab energies.	90:average polarisation potential.
91:polarisation potl for rereading.	201:Separate cross sections.
202:Separate cross sections.	203:Separate cross sections.
204:Separate cross sections.	205:Separate cross sections.
206:Separate cross sections.	207:Separate cross sections.
208:Separate cross sections.	

The general data will be in `output.fro` which if you're using any of my scripts will be `om.fro` and the input file will be `om.fri`

3.2.1 Extracting Tables of cross sections

Depending on what you want to do with FRESKO, this may be your interest so you can plot it. Therefore, if you want to use a program like GNUplot, ROOT, XMGrace, etc., to plot it, you have to clean up the data. This can be done either manually, or by downloading and compiling the following C++ Program

Code 3.5: Program to pull cross sections from fort.16 file with cross sections

```

1  // -----
2  //
3  // conv_fort.cpp
4  //
5  // Written : 18 July 2007 by Brian Roeder, LPC Caen
6  // email   : roeder@lpccaen.in2p3.fr
7  //
8  // Description : A C++ program that reads the Fresco DWBA output file
9  //               "fort.16" and converts it into a text file for use with
10 //               xmgrace or root. Setup only to read differential cross
11 //               sections and not the analyzing powers.
12 //
13 // -----
14
15 #include <iostream>
16 #include <iomanip>
17 #include <fstream> // for file io
18
19 using namespace std;
20
21 int main()
22 {
23     char NewFileName[51];
24
25     cout << "This program converts the fort.16 file into a text file" << endl;
26
27     cout << "Enter the name of the new text file (max 50 characters):" << endl;
28
29     cin >> NewFileName;
30
31     cout << "Converting file fort.16 into angle vs. diff cross section" << endl;
32
33     // First find angular distribution!
34
35     ifstream infile;
36     infile.open("fort.16");
37
38     string dummy;
39
40     if(infile.good())
41     {
42         while(dummy!="END")
43         {
44             infile >> dummy;
45             // cout << dummy << endl;
46         }
47         while(dummy!="projectile")
48         {
49             infile >> dummy;
50             // cout << dummy << endl;
51         }
52     }

```

```

53
54
55 // Now read angular distribution into new text file !!
56
57 ofstream outfile;
58 outfile.open(NewFileName, ios::trunc); // erase old file, if exists
59
60 while(infile.good())
61 {
62     double theta = 0.;
63     double sigma = 0.;
64     double dummy1;
65     double dummy2;
66     double dummy3;
67     double dummy4;
68
69     infile >> theta >> sigma >> dummy1 >> dummy2 >> dummy3 >> dummy4;
70
71     if(sigma > 0.)
72     {
73         outfile << theta << "\t" << sigma << endl;
74         cout << "Theta = " << theta << " , Sigma = " << sigma << endl;
75     }
76     else
77     {
78         cout << "Cross Section at Theta = " << theta << " is zero!!!" << endl;
79     }
80 }
81
82 cout << "Fort.16 file converted to : " << NewFileName << endl;
83 cout << " DONE! " << endl;
84
85 infile.close();
86 outfile.close();
87
88 }
89
90 else
91 {
92     cout << "Error in file reading ! " << endl;
93     cout << "Exiting program! " << endl;
94 }
95 return 0;
96 }

```

Chapter 4

DWUCK4

4.1 Running a DWUCK4 file

To run a file in DWUCK4, it is the same as running a FRESKO file.

```
1  #!/usr/bin/python
2  '''
3  Author: Christopher Mertin
4  Date: July 16, 2015
5
6  This Python script is to parse the theoretical calculations that
   are output
7  from DWUCK4. This script will either plot the data right in front
   of you by using
8  matplotlib, or if you want to access the data after the
   calculation, it will
9  save each calculation into its own separate file. After which you
   can open the
10 files and copy and paste the values into your own plotting
   program
11 (perhaps EXCEL or Calc)
12
13 This file can be downloaded from
14 http://github.com/cmertin/NuclearPrograms/dwuck\_parser.py
15 '''
16
17 from __future__ import print_function
18 import os.path
19 import matplotlib.pyplot as plt
20
21 # Looks in a string array for an index containing a certain
   string
22 def Find(array, string, start=0):
```

```

23     for i in xrange(start, len(array)):
24         if array[i].find(string) != -1:
25             return i
26     return -1
27
28 # Parses the theory file for all the calculated values and their
   titles
29 def ParseTheory(data, theta, x_sec, title):
30     search = "0_Theta_Inelsig_fm**2_Polz_Asy_Ayy
   A22_A21_A20_T22_T21_T20_
   Theta" # Start of calc in DWUCK
31     stop_search = "0Tot-sig" # End of calc string in DWUCK
32     index = Find(data, search)
33     stop_index = Find(data, stop_search, index)
34     while index != -1:
35         title.append(data[index-4])
36         for i in xrange(index+1, stop_index):
37             temp = data[i].split()
38             theta.append(temp[0])
39             x_sec.append(temp[1])
40             index = Find(data, search, index+10)
41             stop_index = Find(data, stop_search, index)
42             theta.append("END") # These make it easier to plot more
   than one
43             x_sec.append("END")
44
45     # Makes all the titles slightly pretier from the output file
46     for i in xrange(0, len(title)):
47         temp = title[i].split()
48         title[i] = temp[0][1:] + "(" + temp[3] + ")" + temp[2]
49
50
51
52 ##### Start of the program #####
53 theta = []
54 x_sec = []
55 titles = []
56 file_data = []
57 num_angles0 = 0
58 num_angles1 = 0
59 filename = raw_input("Input filename: ")
60
61 while os.path.isfile(filename) == False:
62     print("\nInvaild filename. Try again")
63     filename = raw_input("Input filename: ")

```

```

64
65 # Reads in the file with each line as an array index
66 with open(filename) as myfile:
67     file_data = myfile.readlines()
68 # Cleans up the whitespace
69 for i in xrange(0, len(file_data)):
70     file_data[i] = file_data[i].strip()
71
72 ParseTheory(file_data, theta, x_sec, titles)
73 num_angles1 = Find(theta, "END")
74
75 plot_q = raw_input("Would you like to plot the data? If no, will
    write it\
76 to files_(Y/N): ")
77
78 while plot_q != "y" and plot_q != "Y" and plot_q != "n" and
    plot_q != "N":
79     print("\nInvalid response. Try again.")
80     plot_q = raw_input("Would you like to plot the data? If no,
    will write it to files_(Y/N): ")
81
82 # Plots the file by using matplotlib
83 if plot_q == "Y" or plot_q == "y":
84     for i in xrange(0, len(titles)):
85         S = num_angles0 # Start of the data array for this single
            plot
86         E = num_angles1 # End of the data array for this single
            plot
87         plt.semilogy(theta[S:E], x_sec[S:E])
88         plt.rc("text", usetex=True)
89         plt.rc("font", family="serif")
90         plt.title(titles[i], fontsize=25)
91         plt.xlabel(r"$\theta_{c.m.}$", fontsize=18)
92         plt.ylabel(r"$\frac{d\sigma}{d\Omega}$", fontsize=22)
93         plt.show()
94         num_angles0 = num_angles1 + 1
95         num_angles1 = Find(theta, "END", num_angles1+1)
96
97 # Writes all theoretical calculations to their own file with the
    first
98 # line being the title of the calculation that was done
99 else:
100     extension_index = Find(filename, ".")
101     outfile_temp = filename[0:extension_index]
102     extension = filename[extension_index:]

```

```

103     for i in xrange(0, len(titles)):
104         S = num_angles0
105         E = num_angles1
106         outfile = outfile_temp + "_" + str(i) + extension
107         with open(outfile, "w") as myfile:
108             myfile.write(titles[i] + "\n")
109             for j in xrange(S, E):
110                 myfile.write("%0.2f" % float(theta[j]) + "\t%0.5E
"
111                                     % float(x_sec[j]) + "\n")
112         print("Wrote the data for \" " + titles[i] + "\" to \" " +
outfile + "\"")
113         num_angles0 = num_angles1 + 1
114         num_angles1 = Find(theta, "END", num_angles1+1)

```
